

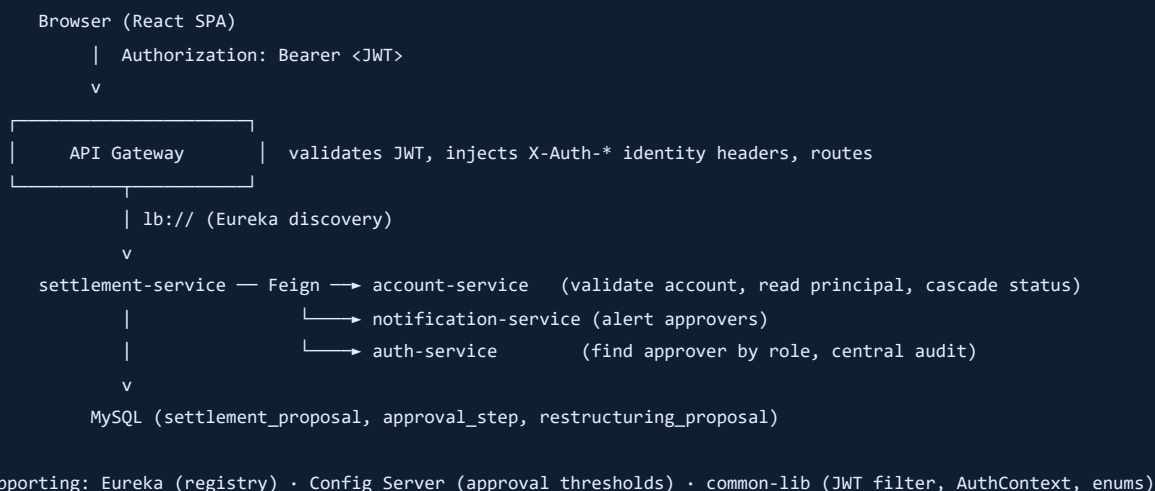
DebtPulse — End-to-End Flow: Settlement & Restructuring

Purpose. A complete walkthrough of the Settlement & Restructuring module — from login, through *creating* a settlement proposal (the "report" action), *submitting* it into a multi-level approval chain, *approving* it, and finally *marking it paid* (which settles the account). Restructuring proposals follow at the end. Every step shows the exact data posted, what comes back, the real code, and the files the data flows through in order.

Audience. Written to be read two ways — a business framing for the Project Manager and a technical framing for a Senior Developer (see the last section).

Why this module is a good example. It exercises the whole stack *and* the most interesting business logic in the system: a server-derived, multi-tier **approval chain**, **maker-checker** control, cross-service validation, and lifecycle **cascades** back onto the account.

1. Architecture at a glance



- **settlement-service** owns two aggregates: **SettlementProposal** (with embedded **ApprovalStep** history) and **RestructuringProposal**.
- The **approval matrix** (which levels must sign off) is derived server-side from the haircut and is configurable via Config Server — never sent by the client.
- Lifecycle **cascades**: a **paid** settlement moves the account to **SETTLED**; an **approved** restructuring moves it to **RESTRUCTURED** (Feign → account-service).

2. Cross-cutting mechanics (brief)

Login returns a JWT access token (in memory) + an httpOnly refresh cookie. Every call carries `Authorization: Bearer <jwt>`. The **gateway** is the only JWT validator; it injects trusted `X-Auth-UserId / X-Auth-Role / X-Auth-BranchId / X-Auth-Name` headers, which `common-lib`'s `RoleBasedHeaderFilter` turns back into a Spring Security context in settlement-service — so `@PreAuthorize` and `AuthContext.currentUserId()` work. Service-to-service calls use OpenFeign over Eureka with Resilience4j fallbacks.

```
POST /api/auth/login {email,password} → 200 { token, userId, role: "SETTLEMENT_OFFICER", ... } (+ refresh cookie)
```

The token's `role` claim drives everything below: officers **raise** proposals; L1/L2/L3 approvers **decide**; admins/managers can do both.

3. Flow B — Create a Settlement Proposal (the "report")

3.1 What the client sends / gets back

```

POST /api/settlements
Authorization: Bearer <JWT>

{
  "accountId": "ACC-2026-000001",
  "totalOutstanding": 200000,    // must equal the account's principal outstanding
  "settlementAmount": 150000,   // must be strictly less than the outstanding
  "paymentDeadline": "2026-09-15",
  "notes": "One-time settlement offer"
}

```

```

200 OK
{
  "proposalId": "SET-2026-000012",
  "accountId": "ACC-2026-000001",
  "officerId": "USR-005",
  "totalOutstanding": 200000,
  "settlementAmount": 150000,
  "haircutPercent": 25.00,        // derived: (200000-150000)/200000*100
  "paymentDeadline": "2026-09-15",
  "approvalLevel": "L3",          // derived highest level (25% >= L3 threshold)
  "requiredApprovalChain": ["L1", "L2", "L3"],
  "currentStep": null,            // not routed until submitted
  "status": "DRAFT",
  "approvalSteps": [],
  ...
}

```

The **server derives** `haircutPercent`, the required `approvalLevel` /chain, and sets `officerId` from the JWT and `status = DRAFT`. The client cannot choose the approval level.

3.2 Frontend — the New Settlement form

The Total Outstanding is **auto-filled from the account and locked**, so it can't be an arbitrary number:

```

// pages/settlement/SettlementWorkspace.jsx
const loadOutstanding = async () => {
  const { data } = await accountApi.get(v.accountId.trim());
  set('totalOutstanding', data.principalAmount);    // fill from the account
};
// Total Outstanding field is disabled (read-only); agent only enters the settlement amount.
<Button onClick={loadOutstanding}>Load outstanding</Button>
...
onSubmit={v => settlementApi.create({ ...v,
  totalOutstanding: Number(v.totalOutstanding), settlementAmount: Number(v.settlementAmount) })}

```

```

// api/services.js
export const settlementApi = {
  create: (body) => api.post('/settlements', body),
  submit: (id) => api.patch(`/settlements/${id}/submit`),
  decide: (id, level, body) => api.post(`/settlements/${id}/decide`, body, { params: { level } }),
  markPaid: (id) => api.patch(`/settlements/${id}/mark-paid`),
  list: (params) => api.get('/settlements', { params }),
};

```

3.3 Backend — controller (RBAC + validation)

```
// settlement-service · SettlementController
@RequestMapping("/api/settlements")
@PreAuthorize("hasAnyRole('ADMIN','SETTLEMENT_OFFICER','PORTFOLIO_MANAGER','L1_APPROVER','L2_APPROVER','L3_APPROVER')")
public class SettlementController {

    @PostMapping
    @PreAuthorize("hasAnyRole('ADMIN','SETTLEMENT_OFFICER')") // only officers/admin may raise
    public ResponseEntity<SettlementResponse> create(@Valid @RequestBody SettlementRequest req) {
        return ResponseEntity.ok(settlementService.create(req));
    }
}
```

3.4 Backend — service (guards + derived chain + persist)

```
// settlement-service · SettlementServiceImpl.create
if (!accountClient.accountExists(req.accountId())) throw new ResourceNotFoundException(...); // Feign
assertOfficerOwnsAccount(req.accountId()); // a SETTLEMENT_OFFICER may only act on their own accounts
validateAmounts(req); // see below
// One in-progress settlement per account (DRAFT/PENDING_APPROVAL/APPROVED)
if (repo.existsByAccountIdAndStatusIn(req.accountId(), List.of(DRAFT, PENDING_APPROVAL, APPROVED)))
    throw new BusinessException("An in-progress settlement already exists ...", "SETTLEMENT_IN_PROGRESS");

BigDecimal haircut = computeHaircut(req.totalOutstanding(), req.settlementAmount());
ApprovalLevel highest = approvalPolicy.highestLevel(haircut); // DERIVED, never from the client

SettlementProposal proposal = SettlementProposal.builder()
    .accountId(req.accountId()).officerId(AuthContext.currentUserId())
    .totalOutstanding(req.totalOutstanding()).settlementAmount(req.settlementAmount())
    .haircutPercent(haircut).paymentDeadline(req.paymentDeadline())
    .approvalLevel(highest).notes(req.notes()).status(SettlementStatus.DRAFT).build();
return mapper.toDto(repo.save(proposal));
```

```
// the amount rules
private void validateAmounts(SettlementRequest req) {
    if (req.settlementAmount().compareTo(req.totalOutstanding()) >= 0) // strictly less
        throw new BusinessException("Settlement amount ... must be less than the total outstanding ...", "INVALID_SETTLEMENT");
    AccountDto account = accountClient.getAccount(req.accountId());
    if (account != null && account.principalAmount() != null
        && req.totalOutstanding().compareTo(account.principalAmount()) != 0) // must match the account
        throw new BusinessException("Total outstanding ... must match the account's principal outstanding ...", "OUTSTANDING");
}
```

3.5 The approval matrix (server-derived)

```
// settlement-service · ApprovalPolicy (thresholds from config-server: l2=10%, l3=25%)
haircut < 10% -> [L1]
10% <= haircut < 25% -> [L1, L2]
haircut >= 25% -> [L1, L2, L3]
```

Approvals are **cumulative and sequential** — every level in the chain must approve, in order.

4. Flow C — Submit → Approve → Paid

4.1 Submit (route into the chain)

```
PATCH /api/settlements/SET-2026-000012/submit → status: PENDING_APPROVAL, currentStep: "L1"
```

```
// SettlementServiceImpl.submit
if (proposal.getStatus() != DRAFT) throw new BusinessException("Only DRAFT settlements can be submitted ...");
ApprovalLevel firstStep = approvalPolicy.requiredLevels(proposal.getHaircutPercent()).get(0); // L1
proposal.setStatus(PENDING_APPROVAL);
proposal.setCurrentStep(firstStep);
notifyApprover(firstStep, saved, "awaits your L1 approval"); // Feign → auth (find approver) + notification-service
```

4.2 Decide (each tier, maker-checker)

```
POST /api/settlements/SET-2026-000012/decide?level=L1
{ "decision": "APPROVE", "comments": "ok" }
```

```
// SettlementServiceImpl.decide – the acting tier is the chain's current step; the ?level= must match it
if (proposal.getStatus() != PENDING_APPROVAL) throw ...;
ApprovalLevel actingLevel = proposal.getCurrentStep();
if (level != actingLevel) throw new BusinessException("You selected " + level + " but this settlement is awaiting " + actingLevel);
// Maker-checker: the approver may not be the officer who raised it
if (approverId.equals(proposal.getOfficerId())) throw new UnauthorizedActionException("Maker-checker violation ...");
// The caller must hold the role for the acting level (ADMIN may act on any)
if (!"ADMIN".equals(role) && !approverRoleFor(actingLevel).equals(role)) throw new UnauthorizedActionException(...);

proposal.addApprovalStep(ApprovalStep.builder().approverId(approverId).level(actingLevel)
    .decision(req.decision()).decidedAt(now()).comments(req.comments()).build());

if (req.decision() == REJECT) { proposal.setStatus(REJECTED); proposal.setCurrentStep(null); notify officer; }
else {
    ApprovalLevel next = approvalPolicy.nextLevel(proposal.getHaircutPercent(), actingLevel);
    if (next != null) { proposal.setCurrentStep(next); notifyApprover(next, ...); } // advance L1→L2→L3
    else { proposal.setStatus(APPROVED); proposal.setCurrentStep(null); proposal.setApprovedById(approverId); notify officer; }
}
```

Response after the final approval: `status: "APPROVED"`, `currentStep: null`, and `approvalSteps` now lists each tier's verdict.

4.3 Mark paid (and settle the account)

```
PATCH /api/settlements/SET-2026-000012/mark-paid → status: PAID (+ account → SETTLED)
```

```
// SettlementServiceImpl.markPaid
if (proposal.getStatus() != APPROVED) throw new BusinessException("Only APPROVED settlements can be marked paid ...");
proposal.setStatus(PAID);
cascadeAccountStatus(saved.getAccountId(), AccountStatus.SETTLED); // Feign → account-service PATCH /internal/accounts/{id}/status
```

4.4 Sequence (settlement lifecycle)

```

Officer → New Settlement (Load outstanding) → settlementApi.create
      → Gateway (JWT→X-Auth-*) → settlement-service.create
      |─ account exists? + owns account? (Feign → account-service)
      |─ amount < outstanding & outstanding == principal
      |─ derive haircut → chain [L1..L3]; save DRAFT
Officer → Submit → PENDING_APPROVAL @ L1 → notify L1 approver
L1 → decide?level=L1 APPROVE → advance → @ L2 → notify L2
L2 → decide?level=L2 APPROVE → advance → @ L3 → notify L3
L3 → decide?level=L3 APPROVE → APPROVED → notify officer
Officer → Mark paid → PAID → account-service: account → SETTLED

```

5. Flow D — Restructuring

A restructuring proposal reworks the repayment plan (revised tenure/EMI, waiver). One live plan per account; approving it moves the account to **RESTRUCTURED**.

```

POST /api/restructuring
{ "accountId": "ACC-2026-000008", "revisedTenure": 24, "revisedEmi": 12000, "waiverAmount": 15000, "startDate": "2026-09-01" }
→ 200 { restructureId: "RES-...", status: "DRAFT", ... }

PATCH /api/restructuring/{id}/approve → status: APPROVED (+ account → RESTRUCTURED)
PATCH /api/restructuring/{id}/reject → status: DRAFT (sent back for revision)

```

```

// RestructuringServiceImpl.create
if (!accountClient.accountExists(req.accountId())) throw ...;
assertOfficerOwnsAccount(req.accountId());
// Only one live plan per account (anything not DEFAULTTED)
if (repo.findByAccountId(req.accountId()).stream().anyMatch(r -> r.getStatus() != DEFAULTTED))
    throw new BusinessException("A restructuring plan already exists ...", "RESTRUCTURE_IN_PROGRESS");
assertViable(req); // revisedEmi × revisedTenure >= outstanding - waiver (reads account via Feign)
// tenure is bounded 1..360 months by @Min/@Max on the request

// RestructuringServiceImpl.approve
proposal.setStatus(APPROVED); proposal.setApprovedById(AuthContext.currentUserId());
cascadeAccountStatus(saved.getAccountId(), AccountStatus.RESTRUCTURED); // Feign → account-service

```

Validation on the request contract:

```

// RestructuringRequest
@Min(1) @Max(360) Integer revisedTenure; // 1..360 months – 1500 is rejected
@Positive BigDecimal revisedEmi;
@PositiveOrZero BigDecimal waiverAmount;

```

6. Endpoint reference

Settlement (</api/settlements>)

Method	Path	Auth	Purpose		
POST	/api/settlements	ADMIN / SETTLEMENT_OFFICER	Create a proposal (status DRAFT)		
PATCH	/api/settlements/{id}/submit	ADMIN / SETTLEMENT_OFFICER	DRAFT → PENDING_APPROVAL @ L1		
POST	/api/settlements/{id}/decide?level=L1\	L2\	L3\	L1/L2/L3 approver / ADMIN	Approve or reject the current tier
PATCH	/api/settlements/{id}/mark-paid	officer / approver / ADMIN	APPROVED → PAID (account → SETTLED)		
GET	/api/settlements	module roles	List/paginate proposals		
GET	/api/settlements/{id}	module roles	View one proposal (with approval history)		
GET	/api/settlements/approval-queue	module roles	Proposals awaiting a decision		
GET	/api/settlements/outstanding · /past-deadline	module roles	Operational lists		
PUT	/api/settlements/{id}	ADMIN / SETTLEMENT_OFFICER	Edit a DRAFT proposal		

Restructuring (/api/restructuring)

Method	Path	Auth	Purpose
POST	/api/restructuring	ADMIN / SETTLEMENT_OFFICER	Create a plan (status DRAFT)
PATCH	/api/restructuring/{id}/approve	L1/L2/L3 approver / ADMIN	Approve (account → RESTRUCTURED)
PATCH	/api/restructuring/{id}/reject	L1/L2/L3 approver / ADMIN	Reject (back to DRAFT)
PUT	/api/restructuring/{id}	ADMIN / SETTLEMENT_OFFICER	Edit a DRAFT plan
GET	/api/restructuring · /{id} · /account/{accountId}	module roles	List / view / by-account

7. File-flow summary (in the order data travels)

Create a settlement proposal

#	File	Layer	What to know
1	<code>frontend/src/pages/settlement/SettlementWorkspace.jsx</code>	FE view	New Settlement form; "Load outstanding" fills+locks the outstanding from the account; calls <code>settlementApi.create</code> .
2	<code>frontend/src/components/FormModal.jsx</code>	FE view	Generic modal; surfaces backend <code>fieldErrors</code> / <code>message</code> .
3	<code>frontend/src/api/services.js</code>	FE API	<code>settlementApi.create/submit/decide/markPaid</code> .
4	<code>frontend/src/api/client.js</code>	FE HTTP	Axios; attaches <code>Bearer</code> token; normalises errors.
5	<code>api-gateway/.../JwtAuthenticationGlobalFilter.java</code>	Gateway	Validates JWT, injects <code>X-Auth-*</code> , routes <code>lb://settlement-service</code> .
6	<code>common-lib/.../security/RoleBasedHeaderFilter.java</code>	BE (shared)	Rebuilds the security context so <code>@PreAuthorize</code> works.
7	<code>settlement-service/.../controller/SettlementController.java</code>	BE ctrl	Class + method RBAC; <code>@Valid</code> ; delegates to the service.
8	<code>settlement-service/.../dto/request/SettlementRequest.java</code>	BE DTO	Inbound contract (accountId, amounts, deadline, notes).
9	<code>settlement-service/.../service/impl/SettlementServiceImpl.java</code>	BE svc	Guards (exists, ownership, amounts, one-active), derive haircut, save DRAFT.
10	<code>settlement-service/.../service/ApprovalPolicy.java</code>	BE policy	Derives the required approval chain from the haircut (config thresholds).
11	<code>settlement-service/.../feign/AccountClient.java</code>	BE client	account exists + principal outstanding (Feign → account-service).
12	<code>account-service/.../controller/InternalAccountController.java</code>	BE ctrl	Answers the internal existence/lookup calls.
13	<code>settlement-service/.../entity/SettlementProposal.java</code> (+ <code>ApprovalStep.java</code>)	BE entity	Persisted proposal + embedded approval history; generates <code>SET-...</code> id.
14	<code>settlement-service/.../repository/SettlementProposalRepository.java</code>	BE data	Save/query → MySQL.
15	<code>settlement-service/.../mapper/SettlementMapper.java</code>	BE map	Entity → <code>SettlementResponse</code> .
16	<code>settlement-service/.../dto/response/SettlementResponse.java</code>	BE DTO	Outbound JSON (haircut, chain, currentStep, status, steps).

Submit → approve → paid (the extra files)

#	File	Layer	What to know
1	<code>SettlementController.submit / decide / markPaid</code>	BE ctrl	Lifecycle endpoints; <code>decide</code> takes <code>?level=</code> + <code>ApprovalDecisionRequest</code> .
2	<code>SettlementServiceImpl.submit</code>	BE svc	DRAFT → PENDING_APPROVAL, sets <code>currentStep</code> = first level, notifies approver.
3	<code>dto/request/ApprovalDecisionRequest.java</code>	BE DTO	The verdict body (<code>decision</code> , <code>comments</code>); the tier is the <code>?level=</code> param.
4	<code>SettlementServiceImpl.decide</code>	BE svc	Maker-checker + role-must-match-step; advances chain or marks APPROVED.
5	<code>feign/AuthClient.java</code> + <code>feign/NotificationClient.java</code>	BE client	Find the approver for a level; alert them; central audit.
6	<code>SettlementServiceImpl.markPaid</code>	BE svc	APPROVED → PAID; cascades account → SETTLED via <code>AccountClient</code> .

Restructuring

#	File	Layer	What to know
1	<code>SettlementWorkspace.jsx</code> (Restructuring tab)	FE view	Create / view / edit (DRAFT); approve/reject actions.
2	<code>settlement-service/.../controller/RestructuringController.java</code>	BE ctrl	RBAC per action; create is officer/admin, approve/reject is approver/admin.
3	<code>settlement-service/.../dto/request/RestructuringRequest.java</code>	BE DTO	Bounded tenure (1..360), positive EMI, non-negative waiver.
4	<code>settlement-service/.../service/impl/RestructuringServiceImpl.java</code>	BE svc	Ownership + one-live-plan + viability guards; approve cascades account → RESTRUCTURED.
5	<code>settlement-service/.../mapper/RestructuringMapper.java</code> + <code>dto/response/RestructuringResponse.java</code>	BE map/DTO	Entity → response JSON.

8. How to explain it to each audience

To the Project Manager (business view). "An officer proposes to settle an account for less than what's owed. The system fixes the outstanding to the account's real balance (they can't fudge it), works out the discount, and — based on how big that discount is — automatically decides how many levels of sign-off are needed (small discounts: one approver; large ones: up to three, in order). Each approver signs off in turn, and crucially the person who raised the offer can never approve their own — that's the four-eyes control auditors expect. Once fully approved and paid, the account is automatically marked settled. Restructuring works the same way for revised repayment plans, and an approved plan flips the account to 'restructured'. Everything is recorded, and each approval is stamped with who did it and when."

To the Senior Developer (technical view). " `settlement-service` owns `SettlementProposal` (with an embedded `ApprovalStep` list) and `RestructuringProposal`. `create` runs behind `@PreAuthorize`, validates via a Feign call to account-service (existence, ownership, `totalOutstanding == principalAmount`, offer strictly `<` outstanding, single active proposal), derives the haircut and the approval chain from `ApprovalPolicy` (config-driven thresholds — never client-supplied), and persists as DRAFT. `submit` routes to `currentStep = chain[0]`. `decide(?level=...)` enforces that the acting level equals `currentStep`, applies maker-checker (`approverId != officerId`) and role-matches the tier, records an `ApprovalStep`, then advances `currentStep` or flips to APPROVED. `markPaid` gates on APPROVED and cascades `account → SETTLED` via Feign; restructuring `approve` cascades `→ RESTRUCTURED`. Approver discovery + alerts go through auth/notification services; all writes are audited. Feign runs over Eureka with Resilience4j fallbacks."

Generated for the DebtPulse project. Module documented: Settlement & Restructuring (login → create → approval chain → paid; plus restructuring).